

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-66
Рогожин А. С.
Проверил:
Хацкевич А. С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Ход работы

Вариант 10

1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- * реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- * подобрать параметры генератора (a , c , N , x_0 и др.) так, чтобы добиться максимального периода последовательности $T_{\max}$ (согласно теоретическим условиям из методички);
- * провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")

На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- * реализовать стандартную схему генерации кандидата в простые числа:
 - * сгенерировать случайное p -битовое число n (используя генератор из Этапа 1);
 - * установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
 - * выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;

* интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

```
import math
import time
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chisquare

# =====
# 1. Генератор с псевдослучайным прореживанием (Shrinking Generator)
# =====

class LFSR:
    """Линейный сдвиговый регистр с обратной связью (LFSR)."""
    def __init__(self, polynomial_taps, state):
        self.taps = polynomial_taps
        self.degree = max(polynomial_taps)
        self.mask = (1 << self.degree) - 1
        self.state = state & self.mask
        if self.state == 0:
            self.state = 1

    def next_bit(self):
        feedback = 0
        for tap in self.taps:
            feedback ^= (self.state >> (tap - 1)) & 1
        out = (self.state >> (self.degree - 1)) & 1
        self.state = ((self.state << 1) & self.mask) | feedback
        return out

def find_lfsr_period(taps, initial_state=19):
    """
    Функция определения длины периода LFSR.
    Для примитивного многочлена степени n=5 период должен быть равен 2^5 - 1 = 31.
    """
    lfsr = LFSR(taps, initial_state)
    start_state = lfsr.state
    period = 0

    while True:
        lfsr.next_bit()
        period += 1
        if lfsr.state == start_state:
            break
        # Страховка от заикливания, если период слишком велик
        if period > (1 << max(taps)):
            break

    return period

class ShrinkingGenerator:
    """Генератор прореживания (Shrinking Generator)."""
    def __init__(self, taps_g1, seed_g1, taps_g2=[5, 3], seed_g2=29):
        self.g1 = LFSR(taps_g1, seed_g1) # Основной генератор G1
        self.g2 = LFSR(taps_g2, seed_g2) # Управляющий генератор G2

    def next_bit(self):
        while True:
            bit_g1 = self.g1.next_bit()
            bit_g2 = self.g2.next_bit()
            if bit_g2 == 1:
                return bit_g1

    def generate_bit_stream(self, N):
```

```

        """Генерация последовательности битов (0 и 1)."""
        return [self.next_bit() for _ in range(N)]

# =====
# 2. Функции анализа бинарного потока и визуализации (Этап 1)
# =====

def analyze_bit_stream(bit_seq):
    """Анализ равномерности распределения битов (0 и 1)."""
    count_0 = bit_seq.count(0)
    count_1 = bit_seq.count(1)
    freq = [count_0, count_1]

    mean = float(np.mean(bit_seq))
    var = float(np.var(bit_seq))

    expected = [len(bit_seq) / 2.0, len(bit_seq) / 2.0]
    chi2_stat, p_value = chisquare(f_obs=freq, f_exp=expected)

    return freq, mean, var, chi2_stat, p_value

def autocorrelation(seq, lag):
    seq_arr = np.array(seq)
    mean = np.mean(seq_arr)
    num = np.sum((seq_arr[:-lag] - mean) * (seq_arr[lag:] - mean))
    den = np.sum((seq_arr - mean) ** 2)
    return num / den if den != 0 else 0.0

def plot_all_in_one_window(results, bit_seqs):
    """Отображение гистограмм 0/1 и автокорреляции в 1 окне."""
    n_cases = len(results)
    fig, axes = plt.subplots(2, n_cases, figsize=(15, 8))

    # Первая строка: Гистограммы соотношения 0 и 1
    for idx, (name, freq, _, _, _, _, _) in enumerate(results):
        ax = axes[0, idx]
        bars = ax.bar(['0', '1'], freq, color=['#5B9BD5', '#ED7D31'],
        edgcolor='black', width=0.5)
        ax.set_title(f"Распределение битов:\n{name}", fontsize=10)
        ax.set_ylabel("Количество битов")
        ax.grid(axis='y', alpha=0.7)

        for bar in bars:
            yval = bar.get_height()
            ax.text(bar.get_x() + bar.get_width()/2.0, yval / 2, f"{int(yval)}",
            ha='center', va='center', color='white', fontweight='bold')

    # Вторая строка: Графики автокорреляции
    lags = range(1, 21)
    for idx, ((name, _, _, _, _, _, _), bit_seq) in enumerate(zip(results,
    bit_seqs)):
        ax = axes[1, idx]
        values = [autocorrelation(bit_seq, lag) for lag in lags]
        ax.plot(lags, values, marker='o', color='coral')
        ax.set_title(f"Автокорреляция битов:\n{name}", fontsize=10)
        ax.set_xlabel("Lag (сдвиг битов)")
        ax.set_ylabel("Корреляция")
        ax.set_ylim(-0.2, 1.0)
        ax.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.show()

# =====
# 3. Метод Михалеску, Сито и Тест простоты АКС (Этап 2)
# =====

def generate_candidate_mihailescu(bits, gen):
    p_bits = max(2, bits - 1)
    p = 0
    for i in range(p_bits):

```

```

        if gen.next_bit() == 1:
            p |= (1 << i)

    p |= (1 << (p_bits - 1))
    p |= 1

    return 2 * p + 1

PRIMES = [
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
    73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151,
    157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233,
    239, 241, 251
]

def trial_division_filter(n):
    for p in PRIMES:
        if n == p:
            return True
        if n % p == 0:
            return False
    return True

def multiply_poly(poly1, poly2, r, n):
    result = [0] * r
    for i in range(r):
        if poly1[i] == 0:
            continue
        for j in range(r):
            if poly2[j] == 0:
                continue
            idx = (i + j) % r
            term = (poly1[i] * poly2[j]) % n
            result[idx] = (result[idx] + term) % n
    return result

def poly_pow(base, exponent, r, n):
    result = [0] * r
    result[0] = 1
    cur_base = base
    exp = exponent

    while exp > 0:
        if exp & 1:
            result = multiply_poly(result, cur_base, r, n)
            cur_base = multiply_poly(cur_base, cur_base, r, n)
            exp >>= 1
    return result

def is_prime_simple(n):
    if n < 2:
        return False
    for d in range(2, math.isqrt(n) + 1):
        if n % d == 0:
            return False
    return True

def get_largest_prime_factor(num):
    max_prime = -1
    temp = num
    while temp % 2 == 0:
        max_prime = 2
        temp //= 2
    i = 3
    while i * i <= temp:
        while temp % i == 0:
            max_prime = i
            temp //= i
        i += 2
    if temp > 2:
        max_prime = temp

```

```

    return max_prime

def aks_test(n):
    if n <= 1:
        return False

    for p in PRIMES:
        if n == p:
            return True
        if n % p == 0:
            return False

    max_b = n.bit_length()
    for b in range(2, max_b + 1):
        low, high = 2, n
        while low <= high:
            mid = (low + high) // 2
            res = mid ** b
            if res == n:
                return False
            if res < n:
                low = mid + 1
            else:
                high = mid - 1

    r = 2
    log2n = n.bit_length()

    while r < n:
        if math.gcd(n, r) != 1:
            return False

        if is_prime_simple(r):
            r_minus_1 = r - 1
            q = get_largest_prime_factor(r_minus_1)
            threshold = 4 * math.sqrt(r) * log2n
            if q > threshold:
                exp = r_minus_1 // q
                if pow(n, exp, r) != 1:
                    break

        r += 1

    if r == n:
        return True

    limit = int(2 * math.sqrt(r) * log2n)
    for a in range(1, limit + 1):
        base_poly = [0] * r
        base_poly[1] = 1
        base_poly[0] = (n - a) % n

        left_poly = poly_pow(base_poly, n, r, n)

        right_poly = [0] * r
        right_poly[0] = (n - a) % n
        n_mod_r = n % r
        right_poly[n_mod_r] = (right_poly[n_mod_r] + 1) % n

        if left_poly != right_poly:
            return False

    return True

def generate_prime_mihailescu_aks(bits, gen):
    start = time.time()
    candidates = 0
    sieved_out = 0

    n = generate_candidate_mihailescu(bits, gen)

    while True:

```

```

        candidates += 1

        if not trial_division_filter(n):
            sieved_out += 1
            n += 2
            continue

        if aks_test(n):
            end = time.time()
            return n, candidates, sieved_out, (end - start)

    n += 2

def run_tests_for_report():
    print("\n=== ТЕСТЫ КОРРЕКТНОСТИ ДЛЯ ОТЧЕТА ===")
    print("Проверка работы теста AKS:")

    test_prime = 31
    is_p = aks_test(test_prime)
    print(f" Тест для {test_prime} (известное простое): {is_p} (Ожидается: True -> {'OK' if is_p else 'ОШИБКА'})")

    test_comp = 35
    is_c = aks_test(test_comp)
    print(f" Тест для {test_comp} (известное составное): {is_c} (Ожидается: False -> {'OK' if not is_c else 'ОШИБКА'})")
    print("=====\n")

# =====
# 4. Главная программа
# =====

if __name__ == "__main__":
    run_tests_for_report()

    N_bits = 10000

    params = [
        ("1.  $f(x) = x^5 + x^2 + 1$ ", [5, 2]),
        ("2.  $f(x) = x^5 + x^4 + x^3 + x^2 + 1$ ", [5, 4, 3, 2]),
        ("3.  $f(x) = x^5 + 1$ ", [5])
    ]

    results = []
    bit_seqs = []

    print("=== ЭТАП 1 – Демонстрация свойств ПСЧ и проверка периода ===")
    for name, taps in params:
        degree = max(taps)
        max_possible_period = (1 << degree) - 1

        # Вычисление периода многочлена
        actual_period = find_lfsr_period(taps)
        is_primitive = (actual_period == max_possible_period)

        gen = ShrinkingGenerator(taps_g1=taps, seed_g1=19, taps_g2=[5, 3],
                                seed_g2=29)
        bit_seq = gen.generate_bit_stream(N_bits)
        freq, mean, var, chi2, p_val = analyze_bit_stream(bit_seq)

        results.append((name, freq, mean, var, chi2, p_val, actual_period,
                        is_primitive))
        bit_seqs.append(bit_seq)

        print(f"\nМногочлен: {name}")
        print(f" Период регистра G1: {actual_period} из {max_possible_period} максимально возможных")
        print(f" Статус примитивности: {'ПРИМИТИВНЫЙ' if is_primitive else 'НЕПРИМИТИВНЫЙ'}")
        print(f" Количество '0': {freq[0]}, Количество '1': {freq[1]}")
        print(f" Среднее: {mean:.4f} (Теор: 0.5), Дисперсия: {var:.4f} (Теор: ")

```

```

0.25) ")
    print(f" Хи-квадрат: {chi2:.4f}, p-value: {p_val:.4f}")
    if p_val > 0.05:
        print(" -> Распределение битов РАВНОМЕРНОЕ (гипотеза принимается).")
    else:
        print(" -> Распределение битов НЕ РАВНОМЕРНОЕ (гипотеза отклоняется).")

    plot_all_in_one_window(results, bit_seqs)

    print("\n=== ЭТАП 2 – Генерация полных простых чисел (Михалеску  $e=1/2$  + AKS) ===")

    gen_prime = ShrinkingGenerator(taps_g1=[5, 2], seed_g1=19, taps_g2=[5, 3],
    seed_g2=29)

    bit_size = 12
    print(f"Генерация 3-х простых чисел ({bit_size} бит)...")

    header = f"| {'Попытка':<8} | {'Битность':<9} | {'Всего кандидатов':<16} |
{'Отсеяно (сито)':<14} | {'Время (сек)':<11} |"
    separator = "-" * len(header)
    print(separator)
    print(header)
    print(separator)

    primes_found = []
    for i in range(1, 4):
        p, cand, sieved, t = generate_prime_mihailescu_aks(bit_size, gen_prime)
        primes_found.append(p)
        print(f"| {i:<8} | {bit_size:<9} | {cand:<16} | {sieved:<14} | {t:<11.4f} |")

    print(separator)
    print("\nСгенерированные числа (в десятичном виде):")
    for i, p in enumerate(primes_found, 1):
        print(f"{i}: {p} (в бинарном виде: {bin(p)})")

```

=== ЭТАП 1 – Демонстрация свойств ПСЧ и проверка периода ===

Многочлен: 1. $f(x) = x^5 + x^2 + 1$

Период регистра G1: 31 из 31 максимально возможных

Статус примитивности: ПРИМИТИВНЫЙ

Количество '0': 3750, Количество '1': 6250

Среднее: 0.6250 (Теор: 0.5), Дисперсия: 0.2344 (Теор: 0.25)

Хи-квадрат: 625.0000, p-value: 0.0000

-> Распределение битов НЕ РАВНОМЕРНОЕ (гипотеза отклоняется).

Многочлен: 2. $f(x) = x^5 + x^4 + x^3 + x^2 + 1$

Период регистра G1: 31 из 31 максимально возможных

Статус примитивности: ПРИМИТИВНЫЙ

Количество '0': 3750, Количество '1': 6250

Среднее: 0.6250 (Теор: 0.5), Дисперсия: 0.2344 (Теор: 0.25)

Хи-квадрат: 625.0000, p-value: 0.0000

-> Распределение битов НЕ РАВНОМЕРНОЕ (гипотеза отклоняется).

Многочлен: 3. $f(x) = x^5 + 1$

Период регистра G1: 5 из 31 максимально возможных

Статус примитивности: НЕПРИМИТИВНЫЙ

Количество '0': 4000, Количество '1': 6000

Среднее: 0.6000 (Теор: 0.5), Дисперсия: 0.2400 (Теор: 0.25)

Хи-квадрат: 400.0000, p-value: 0.0000

-> Распределение битов НЕ РАВНОМЕРНОЕ (гипотеза отклоняется).

=== ЭТАП 2 – Генерация полных простых чисел (Михалеску $e=1/2 + AKS$) ===

Генерация 3-х простых чисел (12 бит)...

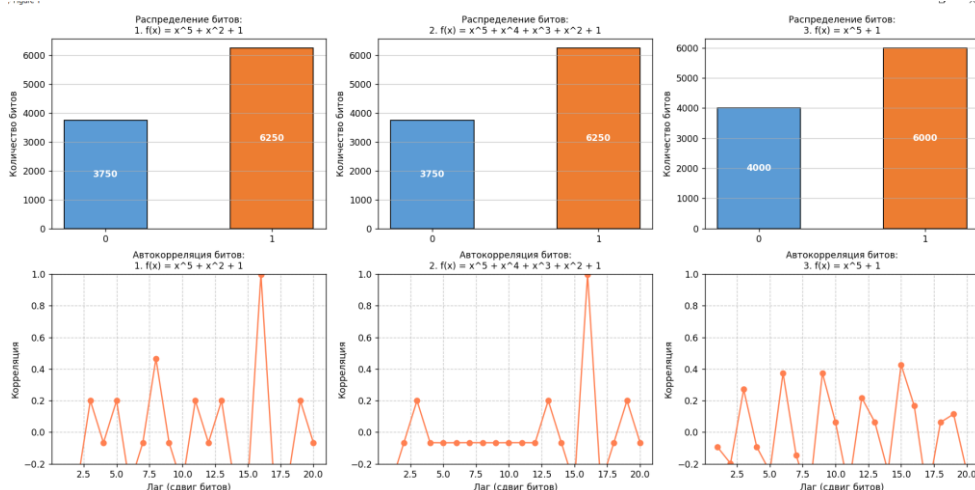
Попытка	Битность	Всего кандидатов	Отсеяно (сито)	Время (сек)
1	12	3	2	0.0079
2	12	4	3	0.0097
3	12	3	2	0.0123

Сгенерированные числа (в десятичном виде):

1: 3511 (в бинарном виде: 0b110110110111)

2: 3709 (в бинарном виде: 0b111001111101)

3: 3803 (в бинарном виде: 0b111011011011)



Вывод: освоил программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей, методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.